

LAPORAN PRAKTIKUM STRUKTUR DATA
PEKAN 7 TENTANG SORTING1



Disusun Oleh :

M.FAJAR FADHILUL ZIKRI

NIM 2311533025

Dosen Pengampu :

DR. WAHYUDI, S.T, M.T

Asisten Praktikum :

SHERLY SUKMADIRA PUTRI

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, TAHUN 2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa atas limpahan rahmat dan karunia-Nya, sehingga laporan praktikum ini dapat diselesaikan tepat pada waktunya. Laporan ini disusun sebagai dokumentasi dan evaluasi dari materi mata kuliah Struktur Data, yang berfokus pada cara mengorganisasi serta mengelola data secara logis dalam sistem komputer.

Praktikum ini memberikan pemahaman mengenai pentingnya pemilihan metode penyimpanan data yang tepat. Struktur data bukan sekadar cara menyimpan informasi, melainkan fondasi dalam membangun algoritma yang efisien. Dengan pengorganisasian data yang baik, sebuah program dapat memproses informasi dengan kecepatan yang lebih tinggi dan penggunaan memori yang lebih optimal.

Melalui rangkaian praktikum ini, mahasiswa diharapkan tidak hanya menguasai teori pengolahan data, tetapi juga mampu mengimplementasikannya dalam bentuk kode program yang sistematis. Kemampuan untuk menyusun data secara terstruktur merupakan keterampilan krusial yang dibutuhkan untuk memecahkan masalah komputasi yang lebih kompleks di masa mendatang.

Terima kasih kami sampaikan kepada dosen pembimbing atas bimbingan dan ilmu yang diberikan selama proses pembelajaran. Terima kasih juga kepada rekan-rekan mahasiswa yang telah memberikan dukungan serta berbagi ide selama praktikum berlangsung. Penyusun menyadari bahwa laporan ini masih memiliki ruang untuk perbaikan, sehingga kritik dan saran yang membangun akan sangat dihargai.

Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan membantu dalam memahami betapa pentingnya peran struktur data dalam dunia pengembangan perangkat lunak.

Padang, 2026

Penyusun

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB 1 PENDAHULUAN.....	1
1.1 Pengertian Praktikum.....	1
1.2 Tujuan Praktikum.....	1
1.3 Persyaratan Praktikum.....	1
1.4 Tempat dan Waktu Praktikum.....	2
BAB 2 PEMBAHASAN PRAKTIKUM.....	3
2.1 Pengertian Sorting (Insertion, Selection, dan Bubble).....	3
2.2 Langkah-Langkah Praktikum.....	3
BAB 3 PENUTUP.....	15
3.1 Kesimpulan.....	15
LAMPIRAN.....	16

BAB 1

PENDAHULUAN

1.1 Pengertian Praktikum

Praktikum Struktur Data adalah kegiatan pembelajaran yang dilakukan di laboratorium komputer untuk mengasah keterampilan mahasiswa dalam memahami serta menerapkan cara pengorganisasian, penyimpanan, dan pengelolaan data secara efisien.

Kegiatan ini tidak hanya menekankan pada penguasaan teori mengenai model-model data, tetapi juga pada latihan implementasi struktur penyimpanan yang optimal, analisis efisiensi algoritma, hingga pengujian performa program. Praktikum dipandang sebagai wahana latihan yang menjembatani pemahaman konseptual mengenai tata kelola data dengan kemampuan teknis dalam memecahkan masalah komputasi yang kompleks secara sistematis.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum ini antara lain sebagai berikut :

1. Membantu mahasiswa memahami konsep dasar struktur data melalui penerapan langsung.
2. Melatih kemampuan menulis, mengompilasi, dan mengeksekusi program dengan mengikuti aturan sintaksis Struktur data.
3. Meningkatkan keterampilan dalam memecahkan masalah (problem solving) dengan pendekatan algoritmik.
4. Membiasakan mahasiswa bekerja sistematis dalam menyusun laporan yang memuat analisis hasil praktikum.
5. Menanamkan sikap teliti, disiplin, serta tanggung jawab dalam melaksanakan kegiatan laboratorium.
6. Mengetahui dan mengaplikasikan Tipe Data stack, queue dll.

1.3 Persyaratan Praktikum

Agar praktikum berjalan lancar, mahasiswa perlu memenuhi beberapa persyaratan berikut:

1. Telah mengikuti perkuliahan teori Struktur Data sebagai dasar pemahaman.
2. Membawa perlengkapan yang diperlukan, antara lain laptop atau komputer yang sudah terpasang Java Development Kit (JDK) dan Integrated Development Environment (IDE) yang direkomendasikan.
3. Mengikuti setiap sesi praktikum sesuai jadwal yang ditetapkan dan hadir minimal sesuai ketentuan program studi.
4. Mematuhi tata tertib laboratorium, termasuk menjaga keamanan data, perangkat, serta lingkungan kerja.
5. Menyusun laporan praktikum dengan format dan aturan yang telah ditetapkan dalam pedoman ini.

1.4 Waktu dan Tempat Praktikum

Pelaksanaan praktikum struktur data mengikuti kalender akademik yang berlaku pada program studi. Setiap sesi praktikum dilaksanakan sesuai jadwal yang ditentukan oleh dosen pengampu. Tempat kegiatan umumnya berlangsung di laboratorium komputer, namun pada kondisi tertentu dapat dilaksanakan secara mandiri dengan perangkat masing-masing, selama memenuhi syarat teknis yang ditetapkan.

BAB 2

PEMBAHASAN PRAKTIKUM

2.1 Pengertian Sorting (Insertion, Selection, dan Bubble)

1. Bubble Sort (Pengurutan Gelembung)

- **Teori Dasar:** Algoritma ini bekerja dengan cara membandingkan dua elemen yang bersebelahan secara berulang-ulang dari awal hingga akhir array, lalu menukarnya (*swap*) jika posisinya salah (misal, angka kiri lebih besar dari angka kanan). Proses ini dianalogikan seperti gelembung sabun, di mana nilai terbesar akan perlahan-lahan "tenggelam" atau bergeser ke posisi paling kanan pada setiap putaran (*pass*).
- **Karakteristik:** Sangat sederhana dan mudah dipahami, namun sangat tidak efisien untuk data dalam jumlah besar.

2. Selection Sort (Pengurutan Pilihan)

- **Teori Dasar:** Konsep utama algoritma ini adalah membagi array menjadi dua bagian: bagian yang sudah terurut (di sebelah kiri) dan bagian yang belum terurut (di sebelah kanan). Pada setiap langkahnya, program akan memindai atau mencari nilai terkecil (atau terbesar) dari bagian yang belum terurut, lalu menukarnya secara langsung dengan elemen pertama dari bagian yang belum terurut tersebut.
- **Karakteristik:** Jumlah pertukaran data (*swap*) dalam algoritma ini sangat minimal dan konstan, karena penukaran hanya terjadi satu kali di setiap akhir putaran.

3. Insertion Sort (Pengurutan Penyisipan)

- **Teori Dasar:** Algoritma ini bekerja mirip seperti cara seseorang merapikan kartu remi di tangan. Data diambil satu per satu dari bagian yang belum terurut, kemudian dibandingkan ke arah kiri dengan elemen-elemen yang sudah terurut. Elemen-elemen yang lebih besar akan digeser ke kanan untuk membuka ruang, lalu elemen baru tersebut akan **disisipkan** ke posisi yang tepat.
- **Karakteristik:** Sangat efisien dan cepat jika kondisi awal datanya sudah hampir terurut secara alami (*nearly sorted*).

Perbandingan Kompleksitas Waktu:

Secara umum, ketiga algoritma ini memiliki kompleksitas waktu rata-rata $O(n^2)$, yang berarti waktu eksekusinya akan meningkat secara kuadratik seiring bertambahnya jumlah data (n). Namun, di antara ketiganya, **Insertion Sort** biasanya memiliki performa dunia nyata yang paling baik untuk data skala kecil karena operasinya yang minim penukaran drastis.

2.7 Langkah Langkah Praktikum

Pada praktikum kali ini kita akan membuat 4 buah class yang pertama Adalah **InsertionSort_2511533023**:

```
public class InsertionSort_2511533023 {
    public static void insertionSort_2511533023(int[] arr) {
        int n_3023 = arr.length;
        for(int i_3023 = 1; i_3023 < n_3023; i_3023++) {
            int key_3023 = arr[i_3023];
            int j_3023 = i_3023 - 1;
            while(j_3023 >= 0 && arr[j_3023] > key_3023) {
                arr[j_3023 + 1] = arr[j_3023];
                j_3023--;
            }
            arr[j_3023 + 1] = key_3023;
        }
    }
    public static void main(String[] args) {
        int arr[] = {23, 78, 45, 8, 32, 56, 1};
        int n_3023 = arr.length;
        System.out.printf("array yang belum terurut:\n");
        for(int i_3023 = 0; i_3023 < n_3023; i_3023++)
            System.out.print(arr[i_3023] + " ");
        System.out.println("");
        insertionSort_2511533023(arr);
        System.out.printf("array yang terurut:\n");
        for(int i_3023 = 0; i_3023 < n_3023; i_3023++)
            System.out.print(arr[i_3023] + " ");
        System.out.println("");
    }
}
```

Hasil Output:

```
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

Class ke-2 adalah SelectionSort_2511533023:

```
public class SelectionSort_2511533023 {
    public static void selectionSort_2511533023(int[] arr) {
        int n_3023 = arr.length;
        for(int i_3023 = 0; i_3023 < n_3023; i_3023++) {
            int minIndex_3023 = i_3023;
            for(int j_3023 = i_3023 + 1; j_3023 < n_3023; j_3023++) {
                if(arr[j_3023] < arr[minIndex_3023]) {
                    minIndex_3023 = j_3023;
                }
            }
        }
    }
}
```

```

        int temp = arr[i_3023];
        arr[i_3023] = arr[minIndex_3023];
        arr[minIndex_3023] = temp;
    }
}

public static void main(String[] args) {
    int arr[] = {23, 78, 45, 8, 32, 56, 1};
    int n_3023 = arr.length;
    System.out.printf("array yang belum terurut:\n");
    for(int i_3023 = 0; i_3023 < n_3023; i_3023++)
        System.out.print(arr[i_3023] + " ");
    System.out.println("");
    selectionSort_2511533023(arr);
    System.out.printf("array yang terurut:\n");
    for(int i_3023 = 0; i_3023 < n_3023; i_3023++)
        System.out.print(arr[i_3023] + " ");
    System.out.println("");
}
}

```

Hasil Output:

```

array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78

```

Class ke-3 BubleSort_2511533023:

```

public class BubleSort_2511533023 {
    public static void bubbleSort_2511533023(int[] arr) {
        int n_3023 = arr.length;
        for(int i_3023 = 0; i_3023 < n_3023; i_3023++) {
            for(int j_3023 = 0; j_3023 < n_3023 - i_3023 - 1; j_3023++) {
                if(arr[j_3023] > arr[j_3023 + 1]) {
                    int temp = arr[j_3023];
                    arr[j_3023] = arr[j_3023 + 1];
                    arr[j_3023 + 1] = temp;
                }
            }
        }
    }

    public static void main(String[] args) {
        int arr[] = {23, 78, 45, 8, 32, 56, 1};
        int n_3023 = arr.length;
        System.out.print("array yang belum terurut:\n");
        for(int i_3023 = 0; i_3023 < n_3023; i_3023++)
            System.out.print(arr[i_3023] + " ");
        System.out.println("");
        bubbleSort_2511533023(arr);
        System.out.print("array yang terurut menggunakan BubleSort:\n");
        for(int i_3023 = 0; i_3023 < n_3023; i_3023++)
            System.out.print(arr[i_3023] + " ");
        System.out.println("");
    }
}

```



```
}
```

Hasil Output:

```
array yang belum terurut:  
23 78 45 8 32 56 1  
array yang terurut menggunakan BubleSort:  
1 8 23 32 45 56 78
```

Class ke-4 InsertionGUI_2511533023:

```
import java.awt.BorderLayout;  
import java.awt.Color;  
import java.awt.Dimension;  
import java.awt.FlowLayout;  
import java.awt.Font;  
  
import javax.swing.BorderFactory;  
import javax.swing.JButton;  
import javax.swing.JFrame;  
import javax.swing.JLabel;  
import javax.swing.JOptionPane;  
import javax.swing.JPanel;  
import javax.swing.JScrollPane;  
import javax.swing.JTextArea;  
import javax.swing.JTextField;  
import javax.swing.SwingConstants;  
import javax.swing.SwingUtilities;  
  
public class InsertionGUI_2511533023 extends JFrame {  
  
    private static final long serialVersionUID = 1L;  
    private int[] array_3023;  
    private JLabel[] labelArray_3023;  
    private JButton stepButton_3023;  
    private JButton resetButton_3023;  
    private JButton setButton_3023;  
    private JTextField inputField_3023;  
    private JPanel panelArray_3023;  
    private JTextArea stepArea_3023;  
  
    private int i_3023 = 1, j_3023;  
    private boolean sorting_3023 = false;  
    private int stepCount_3023 = 1;  
  
    /**  
     * Create the frame.  
     */  
    public InsertionGUI_2511533023() {  
        setTitle("Insertion Sort Langkah per Langkah");  

```

```

        setSize(750,400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        //panel input
        JPanel inputPanel_3023 = new JPanel(new FlowLayout());
        inputField_3023 = new JTextField(30);
        setButton_3023 = new JButton("Set Array");

        inputPanel_3023.add(new JLabel("Masukkan angka (pisahkan dengan koma:"));
        inputPanel_3023.add(inputField_3023);
        inputPanel_3023.add(setButton_3023);

        // Panel array visual
        panelArray_3023 = new JPanel();
        panelArray_3023.setLayout(new FlowLayout());

        // Panel kontrol
        JPanel controlPanel_3023 = new JPanel();
        stepButton_3023 = new JButton ("Langkah Selanjutnya ");
        resetButton_3023 = new JButton ("Reset");
        stepButton_3023.setEnabled(false);
        controlPanel_3023.add(stepButton_3023);
        controlPanel_3023.add(resetButton_3023);

        // Area teks untuk log langkah-langkah
        stepArea_3023 = new JTextArea(8, 60);
        stepArea_3023.setEditable(false);
        stepArea_3023.setFont(new Font("Monospaced", Font.PLAIN, 14));
        JScrollPane scrollPane = new JScrollPane(stepArea_3023);

        // Tambahkan panel ke frame
        add(inputPanel_3023, BorderLayout.NORTH);
        add(panelArray_3023, BorderLayout.CENTER);
        add(controlPanel_3023, BorderLayout.SOUTH);
        add(scrollPane, BorderLayout.EAST);

        // Event Set Array
        setButton_3023.addActionListener(e -> setArrayFromInput());
        stepButton_3023.addActionListener(e -> performStep());
        resetButton_3023.addActionListener(e -> reset());

    }
    private void setArrayFromInput() {
        String text = inputField_3023.getText().trim();
        if (text.isEmpty()) return;
        String[] parts = text.split(",");
        array_3023 = new int[parts.length];
        try {
            for (int k_3023 = 0; k_3023 < parts.length; k_3023++) {
                array_3023[k_3023] = Integer.parseInt(parts[k_3023].trim());
            }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang
dipisahkan "
                + "dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
    }

```

```

        i_3023 = 1;
        stepCount_3023 = 1;
        sorting_3023 = true;
        stepButton_3023.setEnabled(true);
        stepArea_3023.setText("");
        panelArray_3023.removeAll();
        labelArray_3023 = new JLabel[array_3023.length];
        for (int k_3023 = 0; k_3023 < array_3023.length; k_3023++) {
            labelArray_3023[k_3023] = new
JLabel(String.valueOf(array_3023[k_3023]));
            labelArray_3023[k_3023].setFont(new Font("Arial", Font.BOLD, 24));

labelArray_3023[k_3023].setBorder(BorderFactory.createLineBorder(Color.BLACK));
            labelArray_3023[k_3023].setPreferredSize(new Dimension(50, 50));

labelArray_3023[k_3023].setHorizontalAlignment(SwingConstants.CENTER);
            panelArray_3023.add(labelArray_3023[k_3023]);
        }
        panelArray_3023.revalidate();
        panelArray_3023.repaint();
    }
    private void performStep() {
        if (i_3023 < array_3023.length && sorting_3023) {
            int key_3023 = array_3023[i_3023];
            j_3023 = i_3023 - 1;

            StringBuilder stepLog_3023 = new StringBuilder();
            stepLog_3023.append("Langkah ").append(stepCount_3023).
append(": Masukkan ").append(key_3023).append("\n");

            while (j_3023 >= 0 && array_3023[j_3023] > key_3023) {
                array_3023[j_3023 + 1] = array_3023[j_3023];
                j_3023--;
            }
            array_3023[j_3023 + 1] = key_3023;

            updateLabels();
            stepLog_3023.append("Hasil:
").append(arrayToString(array_3023)).append("\n\n");
            stepArea_3023.append(stepLog_3023.toString());

            i_3023++;
            stepCount_3023++;

            if (i_3023 == array_3023.length) {
                sorting_3023 = false;
                stepButton_3023.setEnabled(false);
                JOptionPane.showMessageDialog(this, "Sorting selesai!");
            }
        }
    }

    private void updateLabels() {
        for (int k_3023 = 0; k_3023 < array_3023.length; k_3023++) {
labelArray_3023[k_3023].setText(String.valueOf(array_3023[k_3023]));
        }
    }

    private void reset() {

```

```

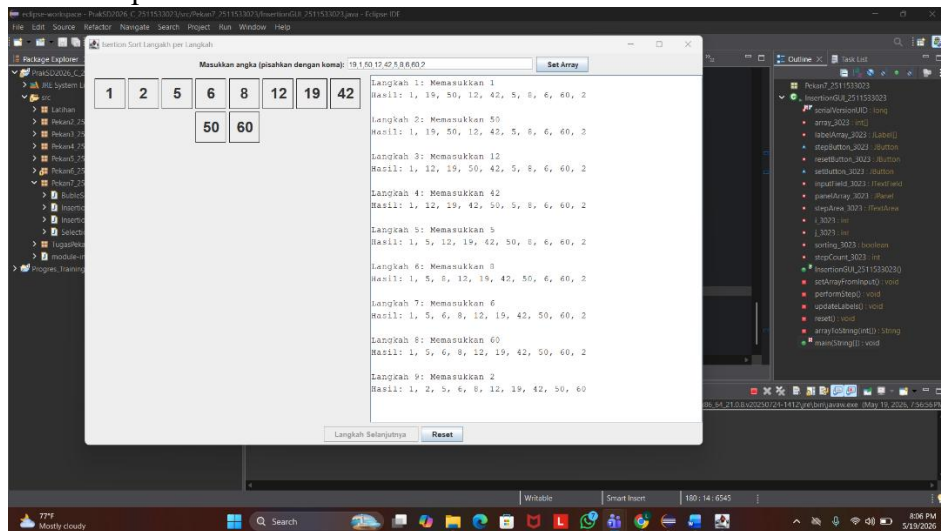
        inputField_3023.setText("");
        panelArray_3023.removeAll();
        panelArray_3023.revalidate();
        panelArray_3023.repaint();
        stepArea_3023.setText("");
        stepButton_3023.setEnabled(false);
        sorting_3023 = false;
        i_3023 = 1;
        stepCount_3023 = 1;
    }

    private String arrayToString(int[] arr) {
        StringBuilder sb_3023 = new StringBuilder();
        for (int k_3023 = 0; k_3023 < arr.length; k_3023++) {
            sb_3023.append(arr[k_3023]);
            if (k_3023 < arr.length - 1) sb_3023.append(", ");
        }
        return sb_3023.toString();
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            InsertionGUI_2511533023 gui = new
InsertionGUI_2511533023();
            gui.setVisible(true);
        });
    }
}

```

Hasil Output:



Program ini berfungsi sebagai simulator visual interaktif berbasis Java Swing untuk mendemonstrasikan cara kerja algoritma *Insertion Sort* secara bertahap (*step-by-step*). Melalui aplikasi ini, pengguna dapat menginput deretan angka acak yang kemudian diubah menjadi kotak-kotak visual di layar, lalu mengontrol jalannya pengurutan menggunakan tombol kendali sembari melihat catatan log perubahan posisi angka secara real-time di kolom riwayat.

BAB 3

PENUTUP

3.1 Kesimpulan

Pada praktikum ini, saya menyimpulkan bahwa pembuatan aplikasi simulator berbasis Java Swing berhasil mempermudah visualisasi proses pengurutan data menggunakan algoritma *Insertion Sort* secara bertahap (*step-by-step*). Melalui praktikum ini, saya dapat memahami secara langsung bagaimana struktur data array dimanipulasi di dalam memori saat elemen disisipkan ke posisi yang tepat, sekaligus membuktikan bahwa pendekatan visual interaktif jauh lebih efektif dalam memperjelas logika pergeseran indeks algoritma dibandingkan hanya melihat output teks statis pada konsol.

Tugas Sorting1:

Mahasiswa_2511533023:

```
public class Mahasiswa_2511533023 {  
  
    private String nama_3023;  
    private String nim_3023;  
    private String prodi_3023;  
  
    // Constructor  
    public Mahasiswa_2511533023(String nama_3023, String nim_3023, String  
prodi_3023) {  
        this.nama_3023 = nama_3023;  
        this.nim_3023 = nim_3023;  
        this.prodi_3023 = prodi_3023;  
    }  
  
    // Getter dan Setter  
    public String getNama_3023() {  
        return nama_3023;  
    }  
  
    public void setNama_3023(String nama_3023) {  
        this.nama_3023 = nama_3023;  
    }  
  
    public String getNim_3023() {  
        return nim_3023;  
    }  
  
    public void setNim_3023(String nim_3023) {  
        this.nim_3023 = nim_3023;  
    }  
  
    public String getProdi_3023() {  
        return prodi_3023;  
    }  
  
    public void setProdi_3023(String prodi_3023) {  
        this.prodi_3023 = prodi_3023;  
    }  
  
    public String toString() {  
        return nama_3023;  
    }  
}
```

Class ini berfungsi sebagai kerangka / blueprint yang akan dipanggil oleh class MainGUI_2511533023 sebagai driver.

Class kedua SortingEngine_2511533023:

```
import java.util.ArrayList;

public class SortingEngine_2511533023 {

    // 1. Algoritma Insertion Sort (Menggunakan format kata 'Langkah')
    public static ArrayList<String> insertionSort(Mahasiswa_2511533023[] arr_3023)
    {
        ArrayList<String> logs_3023 = new ArrayList<>();
        int n_3023 = arr_3023.length;

        logs_3023.add("=== INSERTION SORT ===");

        for (int i_3023 = 1; i_3023 < n_3023; i_3023++) {
            Mahasiswa_2511533023 key_3023 = arr_3023[i_3023];
            int j_3023 = i_3023 - 1;

            // Membandingkan string menggunakan compareToIgnoreCase()
            while (j_3023 >= 0 &&
arr_3023[j_3023].getNama_3023().compareToIgnoreCase(key_3023.getNama_3023()) > 0)
            {
                arr_3023[j_3023 + 1] = arr_3023[j_3023];
                j_3023 = j_3023 - 1;
            }
            arr_3023[j_3023 + 1] = key_3023;

            logs_3023.add("Langkah " + i_3023 + ": " + arrayToString(arr_3023));
        }
        return logs_3023;
    }

    // 2. Algoritma Selection Sort (Menggunakan format kata 'Pass')
    public static ArrayList<String> selectionSort(Mahasiswa_2511533023[] arr_3023)
    {
        ArrayList<String> logs_3023 = new ArrayList<>();
        int n_3023 = arr_3023.length;

        logs_3023.add("=== SELECTION SORT ===");

        for (int i_3023 = 0; i_3023 < n_3023 - 1; i_3023++) {
            int minIdx_3023 = i_3023;
            for (int j_3023 = i_3023 + 1; j_3023 < n_3023; j_3023++) {
                if
(arr_3023[j_3023].getNama_3023().compareToIgnoreCase(arr_3023[minIdx_3023].getNama_3023()) < 0) {
                    minIdx_3023 = j_3023;
                }
            }
            // Proses Swap data objek
            Mahasiswa_2511533023 temp_3023 = arr_3023[minIdx_3023];
            arr_3023[minIdx_3023] = arr_3023[i_3023];
            arr_3023[i_3023] = temp_3023;

            logs_3023.add("Pass " + (i_3023 + 1) + ": " +
arrayToString(arr_3023));
        }
    }
}
```

```

        return logs_3023;
    }

    // 3. Algoritma Bubble Sort (Menggunakan format kata 'Pass')
    public static ArrayList<String> bubbleSort(Mahasiswa_2511533023[] arr_3023) {
        ArrayList<String> logs_3023 = new ArrayList<>();
        int n_3023 = arr_3023.length;

        logs_3023.add("=== BUBBLE SORT ===");

        for (int i_3023 = 0; i_3023 < n_3023 - 1; i_3023++) {
            boolean swapped_3023 = false;
            for (int j_3023 = 0; j_3023 < n_3023 - i_3023 - 1; j_3023++) {
                if
(arr_3023[j_3023].getNama_3023().compareToIgnoreCase(arr_3023[j_3023 +
1].getNama_3023()) > 0) {
                    Mahasiswa_2511533023 temp_3023 = arr_3023[j_3023];
                    arr_3023[j_3023] = arr_3023[j_3023 + 1];
                    arr_3023[j_3023 + 1] = temp_3023;
                    swapped_3023 = true;
                }
            }

            logs_3023.add("Pass " + (i_3023 + 1) + ": " +
arrayToString(arr_3023));
            if (!swapped_3023) break;
        }
        return logs_3023;
    }

    // Helper mengubah data array mahasiswa menjadi format string cetak nama
    private static String arrayToString(Mahasiswa_2511533023[] arr_3023) {
        StringBuilder sb_3023 = new StringBuilder("[");
        for (int i_3023 = 0; i_3023 < arr_3023.length; i_3023++) {
            sb_3023.append(arr_3023[i_3023].getNama_3023());
            if (i_3023 < arr_3023.length - 1) {
                sb_3023.append(", ");
            }
        }
        sb_3023.append("]");
        return sb_3023.toString();
    }
}

```

Class ini berisi method yannga akan di panggil oleh class MainGUI_2511533023 yang mana terdapat 3 metode sorting insertion sort, selection sort, dan bubble sort

Class ke 3 MainGUI_2511533023:

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.ArrayList;

public class MainGUI_2511533023 extends JFrame {
    // Menyimpan basis data dinamis mahasiswa aktif
    private ArrayList<Mahasiswa_2511533023> listMahasiswa_3023 = new
    ArrayList<>();

    // Komponen GUI
    private JTextField txtNama_3023, txtNim_3023, txtProdi_3023;
    private JButton btnTambah_3023, btnHapus_3023, btnSort_3023;
    private JComboBox<String> cmbAlgoritma_3023;
    private JTable table_3023;
    private DefaultTableModel tableModel_3023;
    private JTextArea txtAreaLog_3023;

    public MainGUI_2511533023() {
        // Setup Window GUI Utama
        setTitle("Praktikum Struktur Data Pekan 7 - NIM: 2511533023");
        setSize(850, 600);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        getContentPane().setLayout(new BorderLayout(10, 10));

        // --- PANEL INPUT DATA (UTARA) ---
        JPanel panelInput_3023 = new JPanel(new GridLayout(4, 2, 5, 5));
        panelInput_3023.setBorder(BorderFactory.createTitledBorder("Form Input
Mahasiswa"));

        panelInput_3023.add(new JLabel(" Nama Mahasiswa:"));
        txtNama_3023 = new JTextField();
        panelInput_3023.add(txtNama_3023);

        panelInput_3023.add(new JLabel(" NIM Mahasiswa:"));
        txtNim_3023 = new JTextField();
        panelInput_3023.add(txtNim_3023);

        panelInput_3023.add(new JLabel(" Program Studi:"));
        txtProdi_3023 = new JTextField();
        panelInput_3023.add(txtProdi_3023);

        btnTambah_3023 = new JButton("Tambah Data");
        btnHapus_3023 = new JButton("Hapus Data Terpilih");
        panelInput_3023.add(btnTambah_3023);
        panelInput_3023.add(btnHapus_3023);

        // --- PANEL TENGAH (TABEL DATA & COMBOBOX) ---
        JPanel panelTengah_3023 = new JPanel(new BorderLayout(5, 5));

        String[] kolom_3023 = {"Nama", "NIM", "Program Studi"};
        tableModel_3023 = new DefaultTableModel(kolom_3023, 0);
        table_3023 = new JTable(tableModel_3023);
        JScrollPane scrollTabel_3023 = new JScrollPane(table_3023);
```

```

scrollTabel_3023.setPreferredSize(new Dimension(400, 200));
panelTengah_3023.add(scrollTabel_3023, BorderLayout.WEST);

// Bagian Dropdown Pilihan Algoritma
JPanel panelKontrol_3023 = new JPanel(new FlowLayout(FlowLayout.LEFT));
panelKontrol_3023.setBorder(BorderFactory.createTitledBorder("Metode
Pengurutan"));

String[] opsiAlgoritma_3023 = {"Insertion Sort", "Selection Sort", "Bubble
Sort"};
cmbAlgoritma_3023 = new JComboBox<>(opsiAlgoritma_3023);
btnSort_3023 = new JButton("Mulai Sorting");

panelKontrol_3023.add(new JLabel("Pilih Algoritma: "));
panelKontrol_3023.add(cmbAlgoritma_3023);
panelKontrol_3023.add(btnSort_3023);
panelTengah_3023.add(panelKontrol_3023, BorderLayout.SOUTH);

// --- PANEL SELATAN (VISUALISASI LOG PROSES) ---
JPanel panelLog_3023 = new JPanel(new BorderLayout());
panelLog_3023.setBorder(BorderFactory.createTitledBorder("Visualisasi
Langkah Demi Langkah"));
txtAreaLog_3023 = new JTextArea(12, 50);
txtAreaLog_3023.setEditable(false);
txtAreaLog_3023.setFont(new Font("Monospaced", Font.PLAIN, 12));
JScrollPane scrollLog_3023 = new JScrollPane(txtAreaLog_3023);
panelLog_3023.add(scrollLog_3023, BorderLayout.CENTER);

// Pasang seluruh komponen ke Frame Utama
getContentPane().add(panelInput_3023, BorderLayout.NORTH);
getContentPane().add(panelTengah_3023, BorderLayout.CENTER);
getContentPane().add(panelLog_3023, BorderLayout.SOUTH);

// --- ACTION LISTENERS ---

// Fungsi Tambah Data Ke Tabel
btnTambah_3023.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        String nama = txtNama_3023.getText().trim();
        String nim = txtNim_3023.getText().trim();
        String prodi = txtProdi_3023.getText().trim();

        if (!nama.isEmpty() && !nim.isEmpty() && !prodi.isEmpty()) {
            Mahasiswa_2511533023 mhs_3023 = new Mahasiswa_2511533023(nama,
nim, prodi);
            listMahasiswa_3023.add(mhs_3023);

            tableModel_3023.addRow(new Object[]{mhs_3023.getNama_3023(),
mhs_3023.getNim_3023(), mhs_3023.getProdi_3023()});

            txtNama_3023.setText("");
            txtNim_3023.setText("");
            txtProdi_3023.setText("");
        } else {
            JOptionPane.showMessageDialog(MainGUI_2511533023.this, "Semua
data input wajib diisi!", "Peringatan", JOptionPane.WARNING_MESSAGE);
        }
    }
}

```

```

});

// Fungsi Hapus Baris Terpilih
btnHapus_3023.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        int barisTerpilih_3023 = table_3023.getSelectedRow();
        if (barisTerpilih_3023 != -1) {
            listMahasiswa_3023.remove(barisTerpilih_3023);
            tableModel_3023.removeRow(barisTerpilih_3023);
        } else {
            JOptionPane.showMessageDialog(MainGUI_2511533023.this, "Pilih
baris di tabel terlebih dahulu.", "Info", JOptionPane.INFORMATION_MESSAGE);
        }
    }
});

// Fungsi Memulai Eksekusi Algoritma Pengurutan
btnSort_3023.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        if (listMahasiswa_3023.size() < 2) {
            JOptionPane.showMessageDialog(MainGUI_2511533023.this, "Isi
minimal 2 data mahasiswa untuk diurutkan.", "Info",
JOptionPane.INFORMATION_MESSAGE);
            return;
        }

        // Data awal yang digunakan identik untuk ketiga algoritma
        Mahasiswa_2511533023[] arraySort_3023 =
listMahasiswa_3023.toArray(new Mahasiswa_2511533023[0]);
        String pilihanAlgoritma_3023 = (String)
cmbAlgoritma_3023.getSelectedItem();
        ArrayList<String> hasilLog_3023 = new ArrayList<>();

        if (pilihanAlgoritma_3023.equals("Insertion Sort")) {
            hasilLog_3023 =
SortingEngine_2511533023.insertionSort(arraySort_3023);
        } else if (pilihanAlgoritma_3023.equals("Selection Sort")) {
            hasilLog_3023 =
SortingEngine_2511533023.selectionSort(arraySort_3023);
        } else if (pilihanAlgoritma_3023.equals("Bubble Sort")) {
            hasilLog_3023 =
SortingEngine_2511533023.bubbleSort(arraySort_3023);
        }

        // Bersihkan log lama, cetak serentak ke GUI JTextArea dan Console
        txtAreaLog_3023.setText("");
        for (String logLine_3023 : hasilLog_3023) {
            txtAreaLog_3023.append(logLine_3023 + "\n");
            System.out.println(logLine_3023);
        }

        // Render ulang tabel utama sesuai hasil akhir urutan (A-Z
Ascending)
        listMahasiswa_3023.clear();
        tableModel_3023.setRowCount(0);
        for (Mahasiswa_2511533023 mhs_3023 : arraySort_3023) {
            listMahasiswa_3023.add(mhs_3023);
        }
    }
});

```

```

        tableModel_3023.addRow(new Object[]{mhs_3023.getNama_3023(),
mhs_3023.getNim_3023(), mhs_3023.getProdi_3023()});
    }
    });
}

// Main Run Method
public static void main(String[] args) {
    SwingUtilities.invokeLater(new Runnable() {
        @Override
        public void run() {
            new MainGUI_2511533023().setVisible(true);
        }
    });
}
}

```

Hasil Output:

```

=== INSERTION SORT ===
Langkah 1: [dika, fajar, rifki, arkan]
Langkah 2: [dika, fajar, rifki, arkan]
Langkah 3: [arkan, dika, fajar, rifki]

=== SELECTION SORT ===
Pass 1: [arkan, dika, fajar, rifki]
Pass 2: [arkan, dika, fajar, rifki]
Pass 3: [arkan, dika, fajar, rifki]

=== BUBBLE SORT ===
Pass 1: [arkan, dika, fajar, rifki]

```

Nama	NIM	Program Studi
arkan	2511533005	informatika
dika	2511533025	informatika
fajar	2511533023	informatika
rifi	2511533009	informatika

Class ini berfungsi sebagai pengelola utama (manager class) sekaligus antarmuka interaktif (GUI) yang mengimplementasikan struktur data array atau ArrayList untuk mengatur data mahasiswa secara dinamis. Kelas ini bertanggung jawab menyimpan objek Mahasiswa_2511533023 dan mengontrol eksekusi tiga algoritma pengurutan,

yaitu Insertion Sort, Selection Sort, dan Bubble Sort. Proses perbandingan data string di dalamnya memanfaatkan method `compareToIgnoreCase()` agar pengurutan nama berjalan secara alfabetis (ascending) tanpa terpengaruh oleh sensitivitas huruf kapital. Selain mengelola manipulasi data, kelas ini juga menyediakan visualisasi proses langkah-demi-langkah baik pada komponen teks GUI maupun pada system console, sehingga alur perubahan indeks data dari ketiga algoritma tersebut dapat dipahami dan dianalisis dengan jelas.